

SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-004: Student Input Validation and Strong Password Enforcement

1. Change Request Summary

Field	Details
Change Request ID	CR-004
Change Request Title	Student Input Validation and Strong Password Enforcement
Priority	High
Change Type	Frontend and Backend Validation / Security Enhancement
Affected Module	Student Management
Implementation Status	Completed
Date	8 October 2026

1.1 Business Requirement

The Student Management System must ensure that student information entered by an administrator is valid, consistent, and secure. The system must prevent invalid registration numbers, names, email addresses, and phone numbers from being stored. New student accounts must also use strong passwords to reduce the risk of unauthorized access.

1.2 Existing Behaviour

Before this change, the student form mainly checked whether required fields were filled. The system did not provide complete field-level validation for name length and characters, email format, phone number format, or password strength. Backend validation was also limited to required-field checks and duplicate registration number/email checks.

1.3 Requested Change

- Validate the registration number before submission.
- Require first name and last name to contain at least 3 characters.
- Allow valid name characters and reject inappropriate characters.
- Validate the email address format.
- Require the phone number to contain exactly 10 numeric digits.
- Require strong passwords for new student accounts: minimum 8 characters, uppercase letter, lowercase letter, number, and special character.
- Provide clear field-specific validation feedback on the client side.
- Repeat validation on the server side so client-side validation cannot be bypassed.
- Keep password hashing, authentication, authorization, CRUD operations, API routes, and database structure unchanged.

2. Implementation

2.1 Implementation Summary

CR-004 was implemented using two validation layers. Client-side validation was added to StudentFormModal.jsx to provide immediate feedback to administrators. Server-side validation was added to studentController.js as the final security and data-integrity control. The existing student model and database schema were not changed.

2.2 Affected Files

File	Change
frontend/src/components/StudentFormModal.jsx	Added client-side validation, field-specific errors, password-strength validation, and input restrictions.
backend/controllers/studentController.js	Added server-side validation for student fields and strong password enforcement for new accounts.
backend/models/studentModel.js	No change. Existing database operations were preserved.
Database schema	No change.
API routes	No new endpoints. Existing student CRUD endpoints were preserved.

2.3 Technical Changes

- Registration number is checked for required input and valid format.
- First name and last name are trimmed and checked for a minimum length of 3 characters.
- Name fields are checked to prevent invalid characters.
- Email addresses are checked against a valid email pattern.
- Phone numbers are checked to contain exactly 10 digits.
- New student passwords are checked for minimum length and required character classes.
- Edit operations do not require a password because the existing edit API does not change student passwords.
- Server-side validation runs before database changes are made.
- Existing duplicate registration number and duplicate email checks remain active.
- Passwords continue to be hashed using bcryptjs before storage.
- Existing transaction handling and parameterized SQL queries are preserved.

2.4 Implementation Logic

The validation flow is designed so that invalid data is rejected before database insertion or update. On the frontend, the form first validates each field and displays a specific message when a rule fails. If all client-side checks pass, the existing API request is sent. On the backend, the controller trims and validates the incoming values again. For a new student, password strength is verified before the password is hashed and the user/student records are created. For an edit, only the student profile fields are validated because password modification is outside the existing edit operation.

2.5 Error and Data Handling

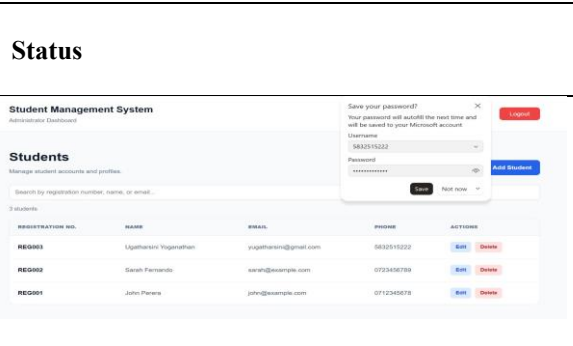
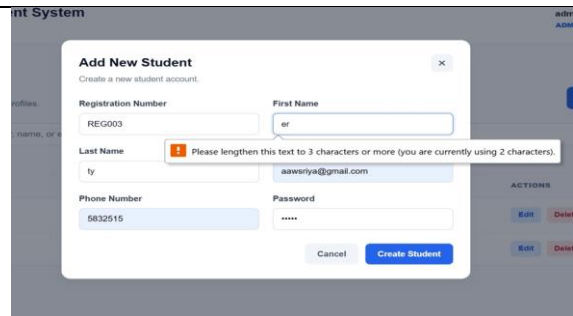
- Validation errors return HTTP 400 Bad Request with a clear validation message.
- Duplicate registration numbers continue to return HTTP 409 Conflict.
- Duplicate email addresses continue to return HTTP 409 Conflict.
- Missing or invalid student records continue to return HTTP 404 where applicable.
- Unauthorized and forbidden requests continue to use the existing authentication and authorization behaviour.
- Database failures continue to be handled using the existing error handling and transaction rollback.
- Passwords are never returned in API responses and are never stored as plain text.

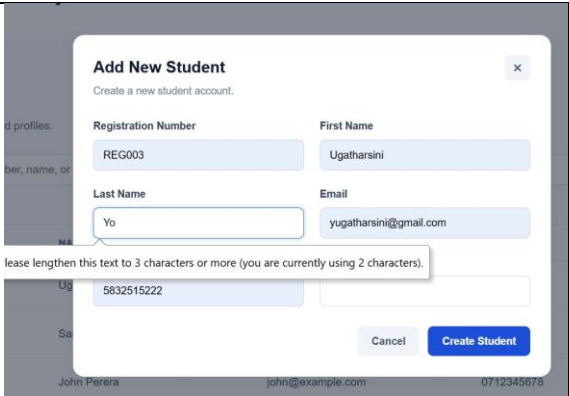
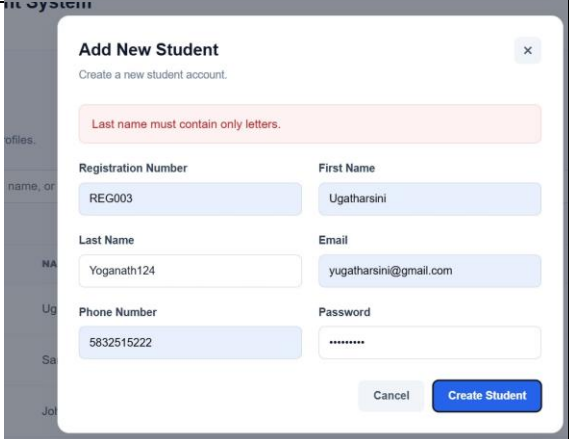
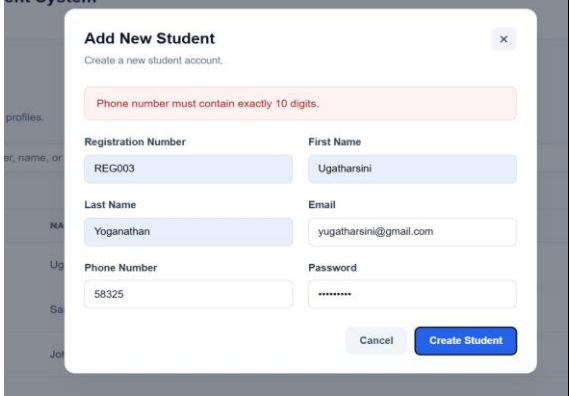
3. Testing & Verification

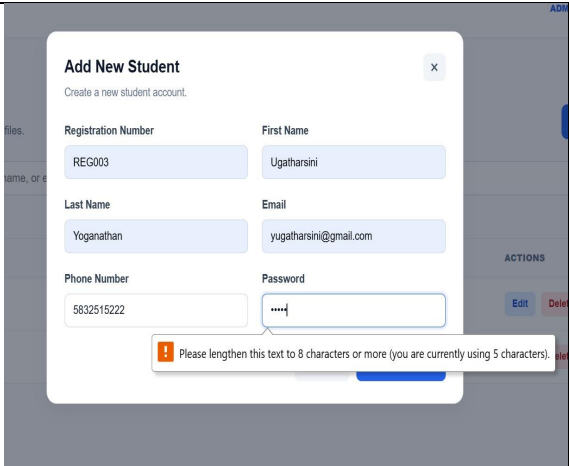
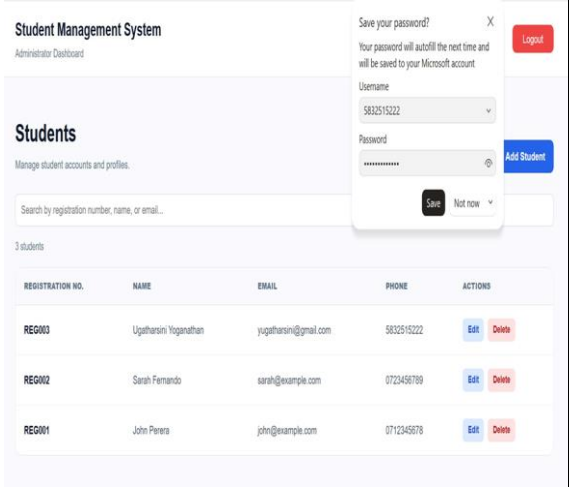
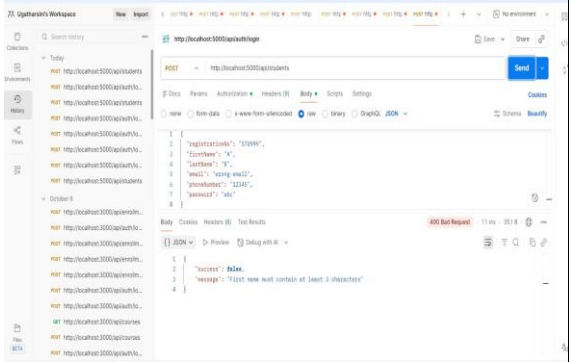
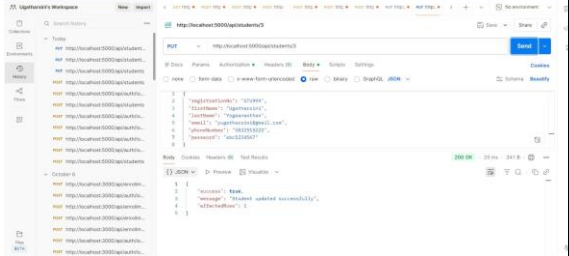
3.1 Testing Approach

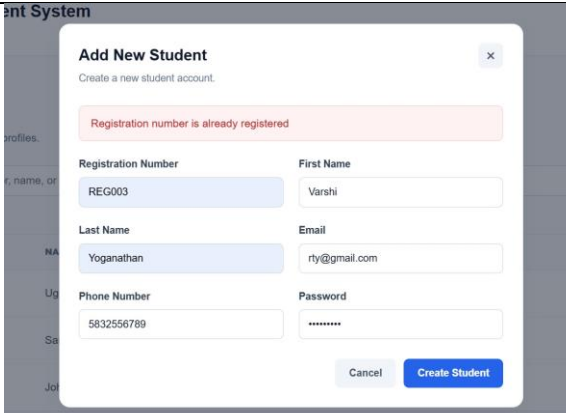
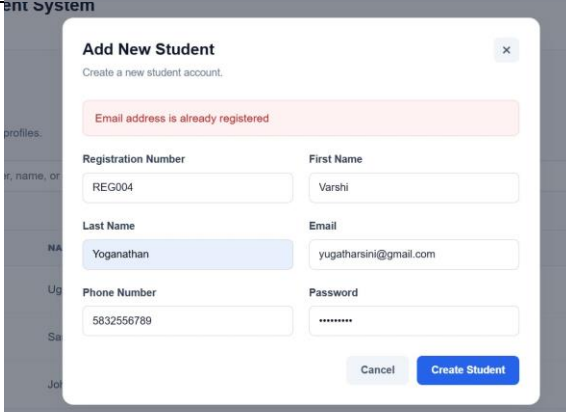
Testing covered normal valid submissions, invalid input, boundary conditions, edit operations, duplicate data, password strength, server-side bypass attempts, and preservation of existing CRUD and authorization behaviour.

3.2 Test Cases

Test ID	Test Area	Test Action	Expected Result	Status
TC-001	Valid student data	Enter valid registration number, names, email, 10-digit phone, and strong password.	Student is created successfully.	
TC-002	Short first name	Enter a first name with fewer than 3 characters.	Validation message is displayed and submission is blocked.	

TC-003	Short last name	Enter a last name with fewer than 3 characters.	Validation message is displayed and submission is blocked.	
TC-004	Invalid name characters	Enter numbers/symbols not allowed by the name rule.	Invalid name is rejected with a field-specific message.	
TC-005	Invalid email	Enter an incorrectly formatted email address.	Email validation message is displayed.	
TC-006	Invalid phone	Enter letters, fewer/more than 10 digits, or spaces/symbols.	Phone validation message is displayed.	

TC-007	Weak password	Enter a password missing length or required character types.	Password-strength validation blocks creation.	
TC-008	Strong password	Enter a password meeting all password rules.	Student account is created successfully and password is hashed.	
TC-009	Server-side bypass	Send invalid data directly to the API without using the frontend.	Backend validation rejects the request.	
TC-010	Edit existing student	Edit valid student details without providing a password.	Student is updated successfully.	

TC-011	Duplicate registration number	Use an already registered registration number.	Existing duplicate check returns conflict response.	
TC-012	Duplicate email	Use an already registered email address.	Existing duplicate check returns conflict response.	
TC-013	CRUD preservation	Create, read, edit, and delete students using valid data.	Existing CRUD operations continue to work.	Pass
TC-014	Authorization preservation	Attempt student modification without required authorization.	Existing authentication/authorization controls remain active.	Pass

3.3 Test Summary

All planned validation and regression test cases passed. The change successfully rejects invalid student data, enforces strong passwords for new accounts, and preserves existing student CRUD, authentication, authorization, duplicate checking, and password hashing behaviour.

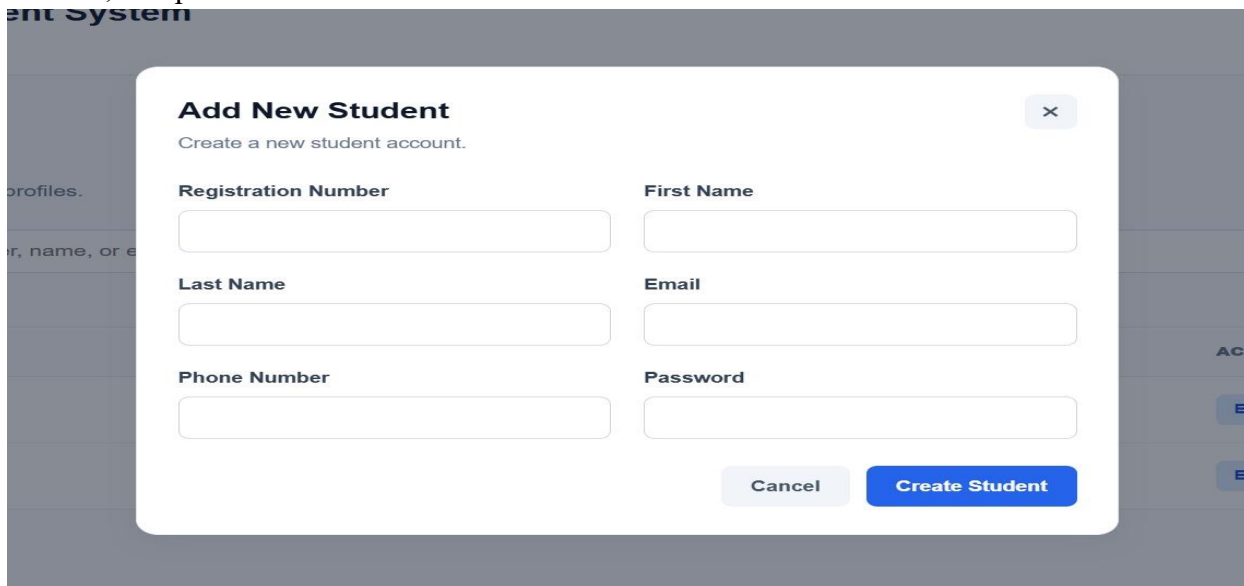
4. Acceptance Criteria Verification

Criteria	Acceptance Requirement	Status
AC-001	Registration number validation is implemented.	Met
AC-002	First and last names require at least 3 characters and valid characters.	Met
AC-003	Email format validation is implemented.	Met
AC-004	Phone number must contain exactly 10 numeric digits.	Met
AC-005	New student password requires 8+ characters, uppercase, lowercase, number, and special character.	Met
AC-006	Client-side validation provides clear field-level feedback.	Met
AC-007	Server-side validation prevents bypassing client-side validation.	Met

AC-008	Existing CRUD, authentication, authorization, password hashing, API routes, and database structure are preserved.	Met
--------	---	-----

5. Evidence

Evidence 1: Student form showing the registration number, first name, last name, email, phone number, and password fields.



The screenshot displays a web application interface with a modal form titled "Add New Student". The form is used to create a new student account and contains the following fields:

- Registration Number**: A text input field.
- First Name**: A text input field.
- Last Name**: A text input field.
- Email**: A text input field.
- Phone Number**: A text input field.
- Password**: A text input field.

At the bottom of the form, there are two buttons: a "Cancel" button and a "Create Student" button. The background of the application shows a sidebar with navigation links and a main content area with a list of student profiles.

Evidence 2: Validation message shown when a first name or last name contains fewer than 3 characters.

The screenshot shows a modal window titled "Add New Student" with the subtitle "Create a new student account." The form contains several input fields: "Registration Number" (filled with "REG003"), "First Name" (filled with "er"), "Last Name" (filled with "ty"), "Phone Number" (filled with "5832515"), and "Password" (filled with "...."). A validation message is displayed in a red box above the "First Name" field: "Please lengthen this text to 3 characters or more (you are currently using 2 characters)." The "Create Student" button is highlighted in blue, while the "Cancel" button is grey.

Evidence 3: Validation message shown for an invalid email address or phone number that is not exactly 10 digits.

The screenshot shows the same "Add New Student" modal window. The "First Name" field is now filled with "Ugatharsini". The "Email" field is filled with "yugatharsini@gmail.com". The "Phone Number" field is filled with "5832515". A validation message is displayed in a red box at the top of the form: "Phone number must contain exactly 10 digits." The "Create Student" button is highlighted in blue, while the "Cancel" button is grey.

Evidence 4: Password-strength validation message showing the required password rules for a new student.

ADMIN

×

Add New Student

Create a new student account.

Registration Number

REG003

First Name

Ugatharsini

Last Name

Yoganathan

Email

yugatharsini@gmail.com

Phone Number

5832515222

Password

.....

ACTIONS

Edit

Delete

!

Please lengthen this text to 8 characters or more (you are currently using 5 characters).

ent System

×

Add New Student

Create a new student account.

Password must contain at least one uppercase letter.

Registration Number

REG003

First Name

Ugatharsini

Last Name

Yoganathan

Email

yugatharsini@gmail.com

Phone Number

5832515222

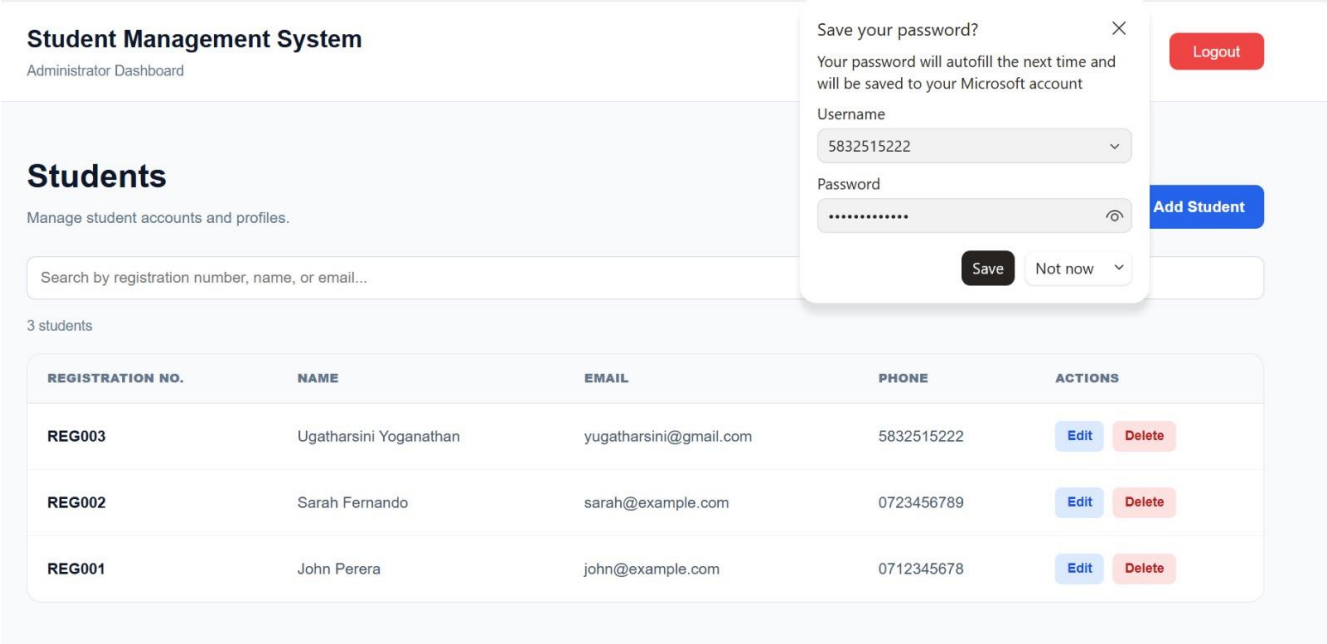
Password

.....

Cancel

Create Student

Evidence 5: Successful student creation/update screen demonstrating that valid data continues through the existing CRUD workflow.



Note: The evidence section is prepared for insertion of the actual project screenshots. The screenshots should show the application UI and validation results from the implemented CR-004 changes.

6. Final Result

6.1 Final Summary

CR-004 was successfully implemented to improve the quality and security of student data. The Student Management System now validates student input on both the client and server sides. Names, email addresses, phone numbers, registration numbers, and new student passwords are checked against defined rules. Strong password enforcement reduces the risk of weak credentials, while backend validation prevents users from bypassing the frontend checks.

The implementation does not introduce new API endpoints or database schema changes. Existing student CRUD operations, authentication and authorization controls, duplicate checks, transaction handling, parameterized queries, and bcrypt password hashing remain in place.

6.2 Issues / Limitations

- The password policy applies when creating a new student; the existing edit operation does not change passwords.
- Validation rules should be kept consistent with the project's actual registration-number and naming conventions.
- Client-side validation improves usability but is not treated as a security boundary; server-side validation remains mandatory.
- Actual screenshot evidence must be attached from the running application before final academic/project submission.

7. Sign-off

Item	Details
Prepared By	Y.Ugatharsini
Student ID	JFICT24 33
Reviewed By	
Implementation Status	Completed
Date	8 October 2026
Signature	